

Supplementary Material

Automated Specification Inference via Heuristic Static Analysis for Program Comprehension and Test Generation

Brendan Edmonds and Mark Utting
University of Queensland

This document provides supplementary material for the main article, including detailed annotation type descriptions, loop invariant heuristic data, a worked inference example, LLM prompt templates, and an approach comparison table.

Contents

1	Appendix A: Generated Annotation Types	1
2	Appendix B: Loop Invariant Inference Heuristics	2
3	Appendix C: Worked Example — End-to-End Inference	3
3.1	Input Method	3
3.2	Precondition Inference (WP-Inspired)	4
3.3	Postcondition Inference (SP-Inspired)	4
3.4	Output: Annotated Method	4
4	Appendix D: LLM Prompt Templates	5
4.1	Phase 1 Prompt (Signature Only)	5
4.2	Phase 2 Prompt (Signature + Guidance)	5
4.3	Phase 3 Prompt (Signature + Specification)	5
4.4	Phase 4 Prompt (Source Code)	5
5	Appendix E: Approach Comparison	5

1 Appendix A: Generated Annotation Types

Unlike traditional JML tools that embed specifications in Javadoc comments, JML-Inferer generates Java annotations that are compiled into bytecode and queryable at runtime via reflection. Table 1 lists the annotation types, all defined in the `com.jml.inferer.annotations` package with runtime retention.

Specifications use JML-like syntax [2, 5] with the following constructs:

- `\result` – The method’s return value
- `\old(expr)` – Value of `expr` at method entry
- `\nothing` – Empty frame (no modifications)
- `this.field` – Instance field reference
- `arr[*]` – All array elements
- `(\forall int k; range; prop)` – Universal quantification

Table 1: Annotation types generated by JML-Inferrer

Annotation	Target	Description
<i>Contract Specifications</i>		
@Requires	Method	Precondition (repeatable)
@Ensures	Method	Postcondition (repeatable)
@Signals	Method	Exceptional postcondition
@LoopInvariant	Method	Loop invariant (repeatable)
@Invariant	Class	Class invariant (repeatable)
@Assignable	Method	Frame condition
<i>Method Classification</i>		
@Pure	Method	No side effects, deterministic
@Observer	Method	Reads but doesn't modify state
@Mutator	Method	Modifies object state
@ThreadSafe	Method/Class	Thread-safe implementation
<i>Nullability</i>		
@Nullable	Field/Param	May be null
@NonNull	Field/Param	Never null
<i>Additional Properties</i>		
@Immutable	Field/Class	Immutable value
@Complexity	Method	Time/space complexity
@MustCall	Method	Required cleanup calls

2 Appendix B: Loop Invariant Inference Heuristics

Loop invariant inference is a key challenge in automated verification [3, 7]. Our tool employs a combination of heuristics that have proven effective in practice.

Heuristic 1: Bound-Based Invariants. For loops with a clear iteration variable and bound:

```
for (int i = 0; i < n; i++) { ... }
```

We infer: $0 \leq i \wedge i \leq n$.

Heuristic 2: Accumulator Patterns. For loops that accumulate values:

```
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
```

We track the accumulator variable and infer bounds based on symbolic unrolling.

Heuristic 3: Monotonicity Detection. We detect whether loop variables are monotonically increasing or decreasing by analysing the update expression:

- $i++$ or $i += k$ (for positive k): monotonically increasing
- $i--$ or $i -= k$ (for positive k): monotonically decreasing

Heuristic 4: Symbolic Unrolling. For loops where pattern-based heuristics fail, we perform bounded symbolic unrolling (default: 3 iterations) to detect emergent patterns. If a consistent relationship is observed, it is generalised to an invariant candidate.

Heuristic 5: While-Loop Counter Detection. For `while` loops, which lack explicit initialization and update expressions, the tool scans the loop body for counter variables—identified by increment/decrement operations (`i++`, `i--`, `i += n`) and compound assignments. Detected counters are then combined with bounds extracted from the loop condition (e.g., `while (i < n)` yields $i \geq 0$ and $i < n$). The same accumulator and variable-relationship analyses used for `for` loops are applied to the body. Common iterator idioms such as `while (iter.hasNext())` are also recognized.

Heuristic 6: Enhanced For-Each Analysis. For enhanced `for-each` loops, the tool infers element membership invariants (`elem` is an element of the iterable) and collection-size bounds (processed elements $\leq$ collection size). Variable-relationship analysis (e.g., max/min tracking) is applied to the loop body, producing richer invariants than element membership alone.

Fallback Behavior. When all heuristics fail, the tool produces a conservative postcondition that does not constrain the loop’s effect. We record this as a “weak specification” for analysis.

Table 2 reports the success rate of each heuristic on the 68 loops encountered across 48 looping methods in our Commons Lang evaluation dataset.

Table 2: Loop invariant inference success rates on the 68 loops of the 48-method looping subset. Multiple heuristics may apply to the same loop; the right-hand column reports the success rate among the loops where the heuristic was applicable.

Heuristic	Applicable loops	Successes	Success rate
Bound-based (for)	32	30	93.8%
Accumulator pattern	15	12	80.0%
Monotonicity detection	13	11	84.6%
While-loop counter	8	6	75.0%
Enhanced for-each	4	3	75.0%
Symbolic unrolling	8	5	62.5%
Overall coverage	68	58	85.3%
Fallback (weak spec)	10	—	—

3 Appendix C: Worked Example — End-to-End Inference

This appendix presents a complete example showing how JML-Inferer analyses a method and derives its specification.

3.1 Input Method

Consider the following utility method that validates and returns an array element:

```
public static String getElement(String[] arr, int index) {
    if (arr == null) {
```

```

        throw new IllegalArgumentException("Array cannot be null");
    }
    if (index < 0 || index >= arr.length) {
        throw new IndexOutOfBoundsException("Invalid index");
    }
    return arr[index];
}

```

Listing 1: Input method before specification inference

3.2 Precondition Inference (WP-Inspired)

The tool identifies guard-and-throw patterns that correspond to preconditions:

1. **First if-statement:** The pattern `if (arr == null) throw ...` is recognised as a null-check guard, yielding precondition `arr != null`.
2. **Second if-statement:** The pattern `if (index < 0 || index >= arr.length) throw ...` is recognised as a range-check guard, yielding preconditions `index >= 0` and `index < arr.length`.

These heuristic inferences correspond to what a formal WP analysis would derive—the weakest precondition for the return statement `return arr[index]` with postcondition `true` is exactly `arr != null ∧ 0 ≤ index < arr.length`.

Inferred Preconditions: `arr != null, index >= 0, index < arr.length`

3.3 Postcondition Inference (SP-Inspired)

The tool analyses the return statement to derive postconditions:

1. The return expression `arr[index]` is an array access on input parameters, yielding `\result == arr[index]`.
2. Since no fields are modified, the method is classified as `@Pure`.

Inferred Postcondition: `\result == arr[index]`

3.4 Output: Annotated Method

```

@Requires("arr != null")
@Requires("index >= 0")
@Requires("index < arr.length")
@Ensures("\result == arr[index]")
@Assignable("\nothing")
@Pure
@Complexity(time = "O(1)", space = "O(1)")
public static String getElement(String[] arr, int index) {
    if (arr == null) {
        throw new IllegalArgumentException("Array cannot be null");
    }
    if (index < 0 || index >= arr.length) {
        throw new IndexOutOfBoundsException("Invalid index");
    }
}

```

```
    return arr[index];  
}
```

Listing 2: Output method with inferred JML specifications

The tool also infers that the method is `@Pure` (no side effects) and has constant time complexity.

4 Appendix D: LLM Prompt Templates

The following prompts were used for LLM-based test generation in each experimental phase. Prompts were kept minimal, differing only in the inclusion of specifications or source code.

4.1 Phase 1 Prompt (Signature Only)

Write unit tests for the following function signature under typical development constraints. Provide the tests as a developer would normally write them, without any formal specification guidance:

[method signature]

4.2 Phase 2 Prompt (Signature + Guidance)

Given the following function signature, write a comprehensive set of unit tests. Ensure that the tests cover normal behavior, edge cases, and potential failure scenarios:

[method signature]

4.3 Phase 3 Prompt (Signature + Specification)

Given the following function signature and formal specification, write a comprehensive set of unit tests. Ensure that the tests cover normal behavior, edge cases, and potential failure scenarios. The specification uses JML notation where `@requires` specifies preconditions and `@ensures` specifies postconditions:

[method signature with JML specification]

4.4 Phase 4 Prompt (Source Code)

Given the following function implementation, write a comprehensive set of unit tests. Ensure that the tests cover normal behavior, edge cases, and potential failure scenarios:

[full method source code]

5 Appendix E: Approach Comparison

Table 3 summarizes the key differences between our approach and related specification inference methods. Key observations:

- Our approach achieves the highest precision among automated methods, due to its conservative heuristic pattern matching grounded in common code idioms.
- LLM-based inference achieves higher recall but lower precision, and lacks determinism.

Table 3: Comparison of specification inference approaches. Precision, recall, and coverage figures for JML-Inferer, Jdoctor, and the LLM-based baseline are computed on the same 312-method corpus and are directly comparable. PreInfer [6] targets a different language family (C#) and could not be applied to the Java corpus; figures are not reported (N/R) because we did not reproduce the tool against an equivalent C# corpus within the scope of this study. Daikon [4] requires an execution infrastructure (test suites or instrumented runs), which is incompatible with the static-only methodology of this article; “varies” indicates that figures published in the original article depend strongly on the choice of test suite.

Approach	Input	Technique	Lang.	Prec.	Rec.	Cov.
JML-Inferer (this work)	Source	Heuristic static	Java	94.2%	89.3%	99.0%
Jdoctor [1]	Javadoc	NLP	Java	92%	83%	45.2%
PreInfer [6]	Execution	Symbolic	C#	N/R	N/R	N/R
Daikon [4]	Traces	Dynamic	Multi	varies	varies	varies
LLM-based (Gemini 2.0)	Source	Neural	Multi	81.3%	88.7%	100%

- Jdoctor achieves high precision but is limited by documentation availability.
- Dynamic approaches like Daikon require execution traces and may miss corner cases not exercised during testing.

References

- [1] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA '18*, pages 242–253. ACM, July 2018. doi: 10.1145/3213846.3213872. URL <http://dx.doi.org/10.1145/3213846.3213872>.
- [2] Patrice Chalin, Joseph R. Kiniry, Gary T. Leavens, and Erik Poll. Beyond assertions: Advanced specification and verification with JML and ESC/Java2. *Formal Methods for Components and Objects*, 4111: 342–363, 2006. doi: 10.1007/11804192_16.
- [3] Patrick Cousot, Radhia Cousot, and Francesco Logozzo. Precondition inference from intermittent assertions and application to contracts on collections. In *International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 150–168, 2011. doi: 10.1007/978-3-642-18275-4_12.
- [4] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.
- [5] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3):1–38, 2006. doi: 10.1145/1127878.1127884.
- [6] Xinyu Wang, Isil Dillig, and Ruzica Piskac. PreInfer: Inferring preconditions via symbolic execution. In *ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 779–793, 2017. doi: 10.1145/3133876.

- [7] Yi Wei, Carlo A. Furia, Nikolay Kazmin, and Bertrand Meyer. Inferring better contracts. In *33rd International Conference on Software Engineering (ICSE)*, pages 191–200, 2011. doi: 10.1145/1985793.1985820.